

Department of Electrical and Computer Engineering

North South University



CSE323.01

Operating Systems Design

Page Replacement Algorithm Simulator

Israt Jahan Sanjana

ID # 2221627642

Faculty Advisor

Safat Siddiqui(SIS)

ECE Department

Spring, 2026

1. Introduction

Memory management is a fundamental concept in operating systems. When physical memory becomes full, the system must decide which pages to remove in order to make space for new ones. This decision is handled by page replacement algorithms such as FIFO (First-In-First-Out), LRU (Least Recently Used), and Optimal.

This project aimed to design and develop a web-based simulator that demonstrates how different page replacement algorithms work. The simulator provides step-by-step visualization of memory frames and allows users to analyze and compare the performance of different algorithms in an interactive way.

2. Objectives

The main objectives of this project are:

- To implement and simulate common page replacement algorithms (FIFO, LRU, Optimal)
- To visualize how pages are loaded and replaced in memory step-by-step
- To clearly identify page hits and page faults (misses)
- To analyze and compare the performance of different algorithms
- To enhance understanding of memory management concepts through visualization

3. Implementation Plan

The project was developed following these phases:

3.1 Requirement Analysis

Understanding the working principles of page replacement algorithms and memory management concepts.

3.2 System Design

Designing the input system, algorithm logic, and output visualization. A web-based architecture was planned with frontend and backend interaction.

3.3 Implementation

Coding the algorithms (FIFO, LRU, Optimal) and integrating them with a Flask backend and a web-based frontend interface.

3.4 Testing

Running multiple test cases with different page sequences and frame sizes to verify correctness.

3.5 Evaluation

Comparing the performance of algorithms based on page faults, hits, and hit ratio through manual analysis by running each algorithm separately.

4. Tools & Technologies

- Programming Language: Python
- Backend Framework: Flask
- Frontend: HTML, CSS, JavaScript
- Development Environment: VS Code

5. Expectations

The expected outcomes of the project were:

- Step-by-step visualization of page replacement
- Clear identification of page hits and faults
- Performance comparison of multiple algorithms
- A user-friendly and interactive interface

6. Technical Report – Challenges Faced (STAR Format)

6.1 Understanding Page Replacement Concepts

Situation:

Initially, understanding how different page replacement algorithms operate within memory management was challenging.

Task:

The goal was to analyze and understand these algorithms during the requirement analysis phase.

Action:

Theoretical concepts of FIFO, LRU, and Optimal algorithms were studied and broken down step-by-step before implementation.

Result:

A strong conceptual understanding was developed, enabling accurate implementation of all algorithms.

6.2 Input Handling and Data Processing

Situation:

The simulator initially misinterpreted user input by treating the entire page sequence as a single value.

Task:

The task was to correctly process user input for page sequences.

Action:

Input parsing logic was improved to handle both space-separated and continuous inputs.

Result:

The system correctly processed page sequences and produced accurate simulation results.

6.3 Implementing Algorithm Logic Correctly

Situation:

Errors occurred in implementing page replacement logic, leading to incorrect page fault calculations.

Task:

The objective was to correctly implement FIFO, LRU, and Optimal algorithms.

Action:

Each algorithm was implemented separately and tested step-by-step to ensure correct memory updates.

Result:

All algorithms functioned correctly and produced reliable outputs.

6.4 Visualizing Output Effectively

Situation:

Initial outputs were displayed as plain text, making it difficult to understand memory behavior.

Task:

The goal was to create a clear and structured visualization system.

Action:

A table-based interface was implemented where rows represent frames and columns represent steps.

Result:

The visualization became more intuitive and aligned with real operating system simulations.

6.5 Identifying Hits and Faults Clearly

Situation:

The system initially displayed only page faults without identifying hits.

Task:

Enhance the simulator to clearly distinguish between hits and misses.

Action:

Hit and miss tracking was added along with color-coded visualization.

Result:

Users could clearly understand algorithm behavior at each step.

6.6 Integrating Multiple Algorithms

Situation:

Initially, only FIFO was implemented, limiting comparison capabilities.

Task:

Add LRU and Optimal algorithms to the system.

Action:

Separate functions were created for each algorithm with a unified output structure.

Result:

Multiple algorithms were successfully integrated into a single system.

7. Future Implementation

Although the current system successfully implements and visualizes page replacement algorithms, several enhancements can be made in the future to further improve functionality and usability.

Firstly, an automatic comparison feature can be added. Currently, users can compare algorithms manually by running them individually. In future development, the system can include a dedicated comparison mode that executes multiple algorithms simultaneously and displays their performance (page faults, hits, and hit ratio) in a single table for easier analysis.

Secondly, the user interface can be made more interactive and user-friendly. Improvements such as better layout design, animations for page replacement steps, and more intuitive controls can enhance the overall user experience.

Additionally, the simulator can be extended to include detailed theoretical explanations of page replacement algorithms. This would allow students to not only simulate algorithms but also understand the underlying concepts directly within the platform. Explanations, examples, and visual guides can be integrated so that users who face difficulty understanding memory management can use this system as a learning resource.

Furthermore, advanced features such as step-by-step animation controls, graphical performance charts, and support for additional algorithms (e.g., LFU, Second Chance) can be incorporated.

Overall, these improvements would transform the project from a simple simulator into a comprehensive educational tool for learning operating system concepts.

8. Conclusion

The project successfully achieved its objectives by implementing and visualizing multiple page replacement algorithms. Through proper system design, implementation, and testing, the simulator provides an interactive and educational tool for understanding memory management. The inclusion of visualization and comparison features enhances its effectiveness and usability, making it a valuable learning resource.

GitHub Page Link -

<https://github.com/Israt-Sanjana/Page-Replacement-Algorithm-Simulator.git>